

AI HealthSecure

Explainable AI and Blockchain-Based Healthcare Disease Prediction, Medical Record Management and Secure Sharing Platform

1. Project Overview

1.1 Project Title

AI HealthSecure: An Explainable Artificial Intelligence and Blockchain-Based Platform for Early Disease Detection, Secure Medical Record Management, Consent-Based Sharing and Integrity Verification

1.2 Project Description

AI HealthSecure is a role-based healthcare management platform that integrates **Artificial Intelligence, Machine Learning, Explainable AI, cryptographic hashing, and blockchain technology** into a single healthcare ecosystem.

The platform is designed around the following core principle:

Predict → Explain → Record → Secure → Authorize → Share → Verify

Doctors use disease-specific machine learning models to analyze patient clinical information and generate disease predictions. The system provides prediction confidence/risk levels and uses SHAP-based explainability to show the factors that contributed to the prediction.

The resulting medical record is stored off-chain in PostgreSQL. A deterministic representation of the record is converted into a **SHA-256 cryptographic fingerprint**, and that fingerprint is registered on a **Solidity smart contract running on a Hardhat network** through Web3.py.

Patients remain in control of record sharing through **patient-generated access tokens**. A token is explicitly associated with a patient, hospital, doctor and expiration period. The authorized doctor can use the token to share a selected patient record with another hospital and doctor.

The receiving hospital or doctor can perform an integrity verification. The application recalculates the SHA-256 hash of the current record and compares it with the original hash registered on the blockchain. If the values match, the record is considered intact. If they differ, the system reports a possible modification.

2. Problem Statement

Healthcare information is often distributed across hospitals, doctors and patients. This creates several practical challenges:

- Patient reports may be stored in different locations.
- Doctors may not have access to previous medical information.
- Patients may need to repeatedly provide the same reports.
- AI predictions may be difficult for doctors to interpret.
- Medical records may be modified after being created or transferred.
- There is a lack of a simple patient-controlled mechanism for sharing records.
- Conventional record systems do not provide an independent integrity fingerprint for detecting modifications.
- Healthcare analytics and model dashboards may rely on static or manually entered data rather than actual application and model results.

AI HealthSecure addresses these problems by combining explainable machine learning with secure digital records and blockchain-based integrity verification.

3. Objectives

The major objectives of the system are:

3.1 Disease Prediction

Use trained machine learning models to support prediction for:

- Diabetes
- Heart Disease
- Kidney Disease
- Liver Disease
- Thyroid Disease

3.2 Explainable AI

Use SHAP to explain which patient features contributed positively or negatively to a model prediction.

3.3 Digital Medical Records

Generate a structured medical record containing:

- Patient details
- Doctor details
- Hospital
- Clinical data

- Reports used for analysis
- Prediction result
- Confidence
- Risk level
- AI analysis
- SHAP explanation
- Model information
- Timestamp

3.4 Secure Medical Record Storage

Store the actual medical information off-chain in PostgreSQL.

3.5 Blockchain-Based Integrity

Generate a SHA-256 fingerprint for each final medical record and register the fingerprint on blockchain.

3.6 Patient-Controlled Sharing

Allow patients to authorize a specific doctor at a specific hospital for a specified period.

3.7 Secure Record Transfer

Allow an authorized doctor to share a selected patient record with another doctor/hospital after token validation.

3.8 Tamper Detection

Allow receiving hospitals/doctors to verify the integrity of records by comparing the current SHA-256 fingerprint with the blockchain-registered fingerprint.

3.9 Dynamic Analytics

Ensure that application charts, dashboards, model metrics and analytics are generated dynamically from real database, prediction and model-training results.

4. Scope of the System

The system covers:

- Role-based authentication
- Patient management
- Doctor management
- Hospital management
- Patient health profiles

- Medical report upload and management
 - Disease-specific clinical-data entry
 - ML prediction
 - Prediction confidence/risk calculation
 - SHAP explainability
 - AI-generated/derived prediction analysis
 - Medical record creation
 - SHA-256 record fingerprinting
 - Blockchain registration
 - Patient consent/access tokens
 - Doctor-to-doctor / hospital-to-hospital record sharing
 - Record integrity verification
 - Blockchain history
 - Verification history
 - Model registry
 - Model-training history
 - Model analysis
 - User management
 - Audit logging
 - Dynamic dashboards and charts
-

5. Users and Role-Based Access

The platform contains four primary roles:

1. Patient
2. Doctor
3. Healthcare Organization / Hospital
4. Administrator

Role-based access control must be implemented both in the frontend and backend.

The frontend controls what the user can see, while the backend must independently enforce authorization.

6. Patient Role

6.1 Patient Responsibility

The patient is primarily the **owner/viewer/controller of their healthcare information**.

The patient does **not** run disease predictions.

The patient does **not** train models.

The patient does **not** manually generate SHAP explanations.

Instead, the patient views the results generated by the doctor.

6.2 Patient Navigation

The patient navigation should contain exactly:

1. Dashboard
2. My Health
3. My Records
4. Medical Reports
5. Access & Sharing
6. Profile

The patient navigation must NOT contain:

- Predict
- Explain
- Integrity
- Model Analysis
- Model Training

These concepts should appear only within the relevant record view when appropriate.

6.3 Patient Dashboard

The dashboard provides a clear overview of the patient's healthcare activity.

KPI cards

- Total Checkups
- Total Records
- Uploaded Reports
- Shared Records
- Active Access Tokens

Recent Checkups

Each checkup should clearly display:

- Checkup date
- Disease predicted
- Prediction result
- Confidence

- Risk
- Doctor
- Hospital
- View Record

Example:

Diabetes — High Risk — 87.4% Confidence

Dr. Rahul Sharma | City General Hospital | 12 Sep 2026

Recent Records

Display the patient's latest medical records.

Recent Activity

Examples:

- New record created
- Medical report uploaded
- Record shared
- Access token generated

All dashboard information must be retrieved dynamically from the database.

7. Patient — My Health

This page contains the patient's health information rather than AI prediction functionality.

Information may include:

- Name
- Age
- Gender
- Height
- Weight
- BMI
- Blood pressure
- Medical history
- Allergies
- Existing conditions

The patient may view and, where permitted, update appropriate profile information.

The page should not attempt to predict disease.

8. Patient — My Records

This is the most important patient feature.

It contains all **doctor-generated medical/checkup records**.

Each record card/table entry should make the disease immediately visible.

Example:

REC-00125
Diabetes
High Risk
87.4% Confidence
Dr. Rahul Sharma
City General Hospital
12 Sep 2026

The disease name must never be hidden behind a generic label such as "Prediction".

8.1 Full Record View

When the patient opens a record, the system should present one comprehensive medical record.

Section 1 — Reports Used

Show the actual reports that were provided to the doctor and used for that particular analysis.

For example:

- Blood Report
- HbA1c Report
- Lab Report

The record should distinguish between:

Reports uploaded by patient

and

Reports actually used in this checkup/prediction

Section 2 — Clinical Data Used

Display the exact clinical values passed to the model for that prediction.

Example:

- Glucose: 168 mg/dL
- HbA1c: 8.2%
- BMI: 29.4
- Age: 45
- Blood Pressure: 130/85 mmHg

These values must be preserved as historical prediction input.

Section 3 — Prediction Report

The prediction result should be the most visually prominent part.

Example:

Predicted Disease: Diabetes

Confidence: 87.4%

Risk Level: High

The page should make the disease name immediately understandable.

Section 4 — AI Analysis

Present a human-readable analysis derived from the actual model output and explanation data.

Section 5 — SHAP Explanation

SHAP should be displayed in an understandable manner.

Possible visualizations:

- SHAP bar chart
- Positive contribution chart
- Negative contribution chart
- Feature contribution graph
- Feature/value/contribution table

Example:

Feature	Patient Value	SHAP Contribution	Effect
Glucose	168 mg/dL	+0.42	Increased Risk
BMI	29.4	+0.21	Increased Risk
Age	45	+0.08	Moderate Effect

The explanation must be generated from the actual SHAP output. The system must not create fictitious explanations.

Section 6 — Doctor Details

Show:

- Doctor name
- Specialization
- Contact/profile information where appropriate

Section 7 — Hospital Details

Show:

- Hospital name
- Location
- Relevant organization information

Section 8 — Blockchain Registration

Show a simple security status such as:

Blockchain Registered

Technical details may be displayed through a secondary/collapsible section:

- SHA-256 hash
- Transaction hash
- Block number
- Registration timestamp
- Smart contract address

The patient should not be overloaded with blockchain terminology.

9. Patient — Medical Reports

This is a separate personal document repository.

The patient can upload reports that they want to keep for future use.

Supported examples:

- Blood reports
- MRI
- CT
- X-Ray
- Prescriptions
- Lab reports
- Other medical documents

Functions:

- Upload
- Preview
- View
- Search
- Filter
- Rename
- Delete where allowed
- View upload date
- View report type

These documents may later be selected by a doctor during a checkup.

This page is distinct from **My Records**.

Difference

Medical Reports = documents the patient stores.

My Records = doctor-generated checkup/AI medical records.

10. Patient — Access & Sharing

Access & Sharing must be a clearly separate navigation item.

The patient creates a consent token.

Generate Token Form

The patient selects:

- Hospital
- Doctor
- Expiry duration in hours

Example:

Hospital A

Doctor A

6 Hours

Then:

Generate Access Token

Example output:

PAT-8F4K-92MX

10.1 Token Association

Every token must be bound to:

```
Patient
+
Authorized Hospital
+
Authorized Doctor
+
Expiration
```

A token must never be treated as a generic patient password.

10.2 Active Tokens

Display:

- Token
- Authorized hospital
- Authorized doctor

- Expiry
- Status
- Created date
- Revoke action where supported

Example:

PAT-8F4K-92MX

Hospital A

Dr. A

Expires 10:30 PM

Active

11. Patient Profile

A profile icon must be present in the navigation/header.

When clicked, the patient should be able to view:

Patient information

- Profile photo
- Patient ID
- Name
- Email
- Phone
- Date of birth
- Gender
- Address

Doctors Who Treated the Patient

Example:

Doctor	Specialization	Hospital	Records
Dr. Rahul Sharma	Cardiologist	City General Hospital	4
Dr. Mehta	Physician	Apollo Hospital	2

This information is dynamically derived from the patient-doctor treatment relationships and medical records.

12. Doctor Role

The doctor is the primary operational healthcare user.

Doctors can:

- Register patients
- Enter clinical information
- Upload/select medical reports
- Run disease analysis
- View predictions
- View confidence/risk
- View SHAP explanations
- Generate medical records
- View patients
- Share selected records
- View received records

Doctors should NOT have:

- Model Training
- Model Analysis

Those functions belong to Admin.

13. Doctor Navigation

The doctor navigation should contain:

1. Overview
 2. Patients
 3. New Patient
 4. New Analysis
 5. Record Sharing
 6. Records
 7. Profile
-

14. Doctor — Overview

The Overview dashboard should provide a complete operational summary.

KPI cards

- Total Patients
 - Total Analyses
 - Today's Predictions
 - Records Created
 - Records Received
 - Pending Shares
-

14.1 Disease-Wise Prediction Chart

Show actual prediction distribution:

- Diabetes
- Heart Disease
- Kidney Disease
- Liver Disease
- Thyroid Disease

Counts must come from the `predictions` table.

14.2 Monthly Analysis Chart

Show the number of actual analyses performed over time.

14.3 Patient Distribution

Show:

- Patients treated by current doctor
 - Patients received through sharing
-

14.4 Recent Patient/Analysis Table

Each row should show:

- Patient name
- Patient ID
- Disease predicted
- Confidence
- Risk

- Date
- Source
- Action

The disease must be clearly visible.

For received patients, show:

Received from: Hospital A / Dr. A

15. Doctor — Patients

Display all patients handled by the doctor.

Filters:

- Patient Name
- Patient ID
- Disease
- Date
- Source

Table:

Patient	Patient ID	Age/Gender	Disease	Last Visit	Source	Action
---------	------------	------------	---------	------------	--------	--------

The source must distinguish:

Treated by me

versus

Received from another hospital/doctor

16. Doctor — New Patient

This begins the primary doctor workflow.

Step 1 — Basic Information

- Patient name
- Date of birth / age

- Gender
- Phone
- Email
- Address

System generates a unique patient ID.

Example:

PAT-1025

Step 2 — Medical Information

- Height
 - Weight
 - BMI
 - Blood pressure
 - Medical history
 - Allergies
 - Existing conditions
-

Step 3 — Medical Reports

Upload or attach relevant reports.

Examples:

- Blood report
- MRI
- CT
- X-Ray
- Lab report
- Prescription

After creation:

Patient Created Successfully

Patient ID:

PAT-1025

Button:

Start New Analysis

17. Doctor — New Analysis

The workflow is:

```
graph TD; A[Select Patient] --> B[Select Disease]; B --> C[Load Disease-Specific Model Configuration]; C --> D[Display Required Features]; D --> E[Enter Clinical Data]; E --> F[Select Relevant Medical Reports]; F --> G[Run Prediction]; G --> H[Prediction Result]; H --> I[SHAP Explanation]; I --> J[AI Analysis]; J --> K[Generate Medical Record];
```

18. Disease-Specific ML Configuration

The frontend must never assume that all diseases use identical input fields.

Each registered model must define:

- Disease
- Model file
- Model version
- Algorithm
- Feature names
- Feature order
- Data type
- Units
- Validation rules

- Preprocessing
- Probability support

When Doctor selects a disease, the application retrieves that configuration and displays the appropriate fields.

This ensures that the exact feature order expected during training is preserved during inference.

19. AI/ML Pipeline

The ML pipeline is:

```
Clinical Data
    ↓
Validation
    ↓
Disease-Specific Preprocessing
    ↓
Trained Model
    ↓
Prediction
    ↓
Probability / Confidence
    ↓
Risk Classification
    ↓
SHAP Explanation
    ↓
AI Analysis
    ↓
Medical Record
```

The application should integrate the already-trained model artifacts rather than pretending to train models during every prediction.

20. Prediction Output

The primary output must clearly state:

Predicted Disease: Diabetes

Confidence: 87.4%

Risk Level: High

The exact presentation can vary according to model output, but the disease must always be visible first.

21. Prediction Data Preservation

The exact model input used for each prediction must be stored.

For example, if a prediction was generated with:

```
Glucose = 168  
HbA1c = 8.2  
BMI = 29.4  
Age = 45
```

those historical values must remain associated with that prediction even if the patient later updates their health profile.

This is essential for reproducibility and auditability.

22. Doctor — Patient Profile

When a doctor clicks a patient, the complete patient profile should become available.

Sections:

- Overview
 - Clinical Data
 - Reports
 - Prediction History
 - Records
 - Sharing
-

22.1 Prediction History

Each prediction should immediately show:

Diabetes — High Risk — 87.4%

rather than a generic "Prediction #103".

22.2 Full Prediction/Record View

Display:

1. Reports used
 2. Exact clinical data used
 3. Predicted disease
 4. Confidence
 5. Risk
 6. AI analysis
 7. SHAP charts
 8. SHAP feature table
 9. Doctor details
 10. Hospital details
 11. Blockchain registration status
-

22.3 Patient Token Within Doctor Patient Profile

If the patient has generated an access token for this doctor/hospital, it should be visible directly inside the relevant patient profile.

Example:

Access to Share

Token: PAT-8F4K-92MX

Hospital: Hospital A

Doctor: Dr. A

Expiry: 10:30 PM

Status: Active

This makes the doctor workflow straightforward.

23. Doctor — Record Sharing

Record Sharing is a dedicated tab.

The screen should ask for:

Step 1

Select receiving hospital.

Step 2

Select receiving doctor.

Step 3

Enter the patient access token.

Step 4

Select the specific patient.

Step 5

Select the specific medical record/file to share.

Step 6

Send record.

24. Token Validation Logic

The backend must verify all of the following:

```
Token exists
    AND
Token belongs to selected patient
    AND
Authorized hospital matches
    AND
Authorized doctor matches
    AND
Token has not expired
    AND
Selected record belongs to selected patient
    ↓
ALLOW SHARE
```

If any condition fails:

Sharing must be blocked.

Example:

This access token is not valid for the selected patient, doctor or hospital.

The frontend must not be trusted to perform this validation by itself.

25. Doctor — Records

This is the doctor's unified record repository.

It contains:

Own Records

Records created by the doctor.

Received Records

Records received from another doctor/hospital.

Search and filter:

- Patient name
- Patient ID
- Disease
- Date
- Source

For received records, explicitly show:

Received from: Hospital A / Dr. A

This provides traceability.

26. Doctor Profile

A profile icon must appear in the navigation/header.

When clicked, it should display:

- Doctor photo
- Doctor name
- Specialization
- Doctor ID

- Email
- Phone
- Hospital
- Hospital address
- Professional information

The system should also show a summary of the doctor's activity where appropriate.

27. Hospital / Healthcare Organization Role

The hospital role focuses primarily on authorized record handling and verification.

Suggested navigation:

1. Dashboard
2. Authorized Records
3. Received Records
4. Shared Records
5. Verify Record
6. Verification History
7. Organization Profile

The organization role is not intended to replace the doctor's clinical workflow.

28. Hospital — Received Records

The hospital can view records received from other healthcare organizations.

Each record should clearly show:

- Patient
 - Disease
 - Doctor
 - Source hospital
 - Received date
 - Record status
 - Verification status
-

29. Hospital — Verify Integrity

The receiving hospital should have a simple action:

Verify Integrity

The backend performs:

```
Current PostgreSQL Record
      ↓
Canonical Representation
      ↓
SHA-256
      ↓
Current Hash
```

Then retrieves:

```
Blockchain
      ↓
Original Registered Hash
```

Then:

```
Current Hash == Blockchain Hash
```

Result:

Match

Record Integrity Verified

Mismatch

Possible Modification Detected

The application should clearly explain that blockchain provides an immutable registered fingerprint; it does not magically prevent changes to off-chain PostgreSQL data. The comparison detects whether the current record differs from the originally registered fingerprint.

30. Administrator Role

Admin has complete system-management visibility.

Admin navigation:

1. Dashboard
 2. User Management
 3. Approvals
 4. Hospitals
 5. Patients
 6. Doctors
 7. AI Model Registry
 8. Model Training
 9. Model Analysis
 10. Medical Records
 11. Blockchain
 12. Verification Logs
 13. Analytics
 14. Profile / Settings
-

31. Admin — Dashboard

The Admin Dashboard provides system-wide statistics.

Examples:

- Total Patients
- Total Doctors
- Total Hospitals
- Total Medical Records
- Total Predictions
- Total Shared Records
- Blockchain Registered Records
- Verified Records
- Verification Failures

All values must be dynamic.

32. Admin — Hospitals

Display hospital cards.

Example:

Hospital A

Clicking a hospital displays:

```
Hospital A
↓
Doctors belonging to Hospital A
↓
Patients handled by each doctor
↓
Records/predictions associated with those patients
```

For each doctor, Admin should be able to view:

- Doctor details
- Patients treated
- Disease predictions
- Medical records
- Reports
- SHAP explanations
- Sharing history

33. Admin — Patients

The Patients page shows all patients.

Clicking a patient should reveal:

- Patient profile
 - Patient ID
 - Health profile
 - All doctors who treated the patient
 - All hospitals where the patient was treated
 - Medical reports
 - Prediction history
 - Prediction reports
 - SHAP explanations
 - Medical records
 - Sharing history
 - Access-token history where appropriate
 - Blockchain details
 - Integrity verification history
-

34. Admin — User Management

Admin must be able to:

- Create user
- Edit user
- Delete/deactivate user
- Activate user
- Approve users where required
- Assign roles
- Search users
- Filter users
- View complete profiles

The functionality must be real and connected to backend APIs.

35. Admin — AI Model Registry

The AI Model Registry contains all supported disease models.

Each model should display:

- Disease
- Algorithm
- Model version
- Feature count
- Preprocessing version
- Training date
- Accuracy
- Precision
- Recall
- F1 score
- ROC-AUC
- Status

The Registry must be connected to actual model metadata.

36. Admin — Model Training

This section stores training history.

A training run can include:

- Dataset name
- Dataset version
- Algorithm
- Training date
- Training samples
- Testing samples
- Features
- Accuracy
- Precision
- Recall
- F1
- ROC-AUC
- Training duration
- Model version
- Artifact path
- Status

If the project does not implement live retraining, the UI should clearly represent this as **training history/model artifacts** rather than offering fake "Train Model" functionality.

37. Admin — Model Analysis

Model Analysis can present actual results such as:

- Confusion matrix
- ROC curves
- Precision/recall analysis
- Class distribution
- Feature importance
- Model comparison
- Performance by disease/model
- Historical model versions

Every visualization must be created from actual stored analysis data.

38. Dynamic Analytics Rule

This is a mandatory system-wide rule:

No static/fake chart values in the final application.

Examples:

Doctor chart:

```
predictions table
  ↓
actual disease counts
  ↓
dynamic chart
```

Admin model chart:

```
model_training_runs
  ↓
actual metrics
  ↓
dynamic chart
```

Patient dashboard:

```
medical_records
predictions
reports
tokens
  ↓
dynamic KPIs
```

If a model has different accuracy from another model, the chart must show that actual difference.

The application must never display the same metric values merely to make the interface look populated.

39. Database Architecture

PostgreSQL is the primary application database.

Major groups:

Authentication

- users

Healthcare entities

- hospitals
- doctors
- patients
- patient_doctor_relationships
- patient_health_profiles

Documents

- medical_reports

AI/ML

- ai_models
- model_features
- model_training_runs
- model_analysis_results
- predictions
- prediction_inputs
- prediction_analysis
- shap_explanations
- shap_features
- prediction_reports

Records

- medical_records

Consent and sharing

- access_tokens
- record_shares

Blockchain

- blockchain_records

Verification

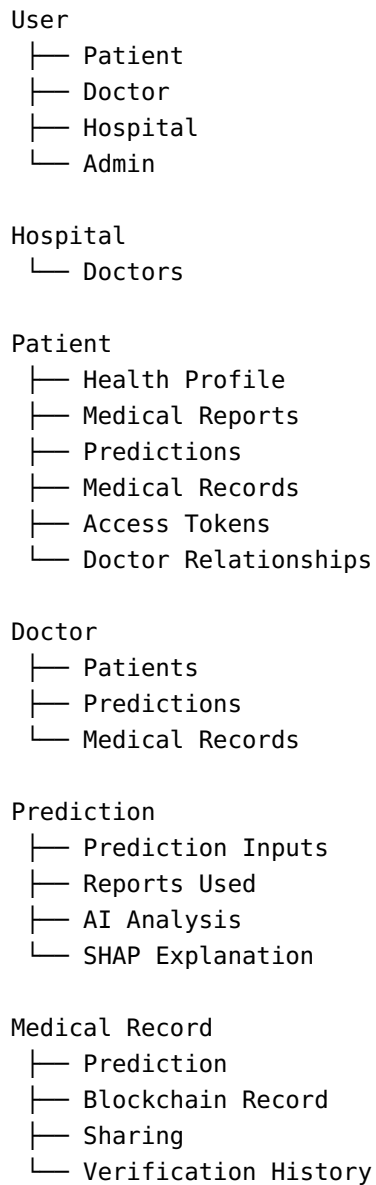
- verification_logs

Auditing

- audit_logs
-

40. Core Database Relationships

The central relationships are:



41. Medical Record Architecture

A final record combines:

```
Patient
+
Doctor
+
Hospital
+
Clinical Data
+
Reports Used
+
Disease
+
Prediction
+
Confidence
+
Risk
+
Model Metadata
+
AI Analysis
+
SHAP Explanation
+
Timestamp
```

The record is stored in PostgreSQL.

42. File Storage

Medical files should not be stored directly as large objects inside normal relational records unless there is a specific requirement.

Recommended application structure:

```
Backend
├── uploads
│   └── medical_reports
```

PostgreSQL stores:

- File name
- File path

- File type
- File size
- Patient
- Uploader
- Report date
- Description

The architecture can later move the file layer to cloud/object storage without redesigning the relational schema.

43. Blockchain Architecture

The blockchain subsystem contains:

- Solidity smart contract
 - Hardhat local network
 - Web3.py integration
 - Blockchain transaction handling
 - Blockchain metadata storage in PostgreSQL
-

44. Blockchain Data Model

The blockchain should store only the information required for integrity registration.

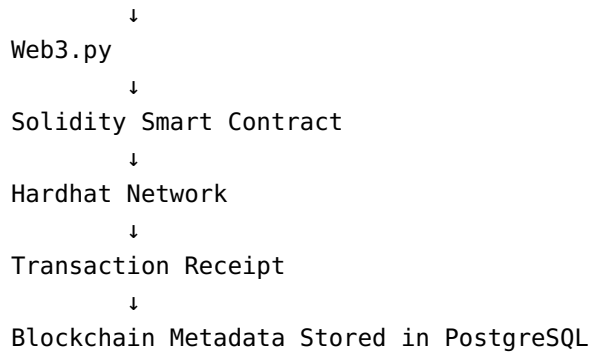
Conceptually:

Record ID
SHA-256 Hash
Timestamp

The complete medical record must remain off-chain.

45. Blockchain Registration Flow

Medical Record Created
↓
Canonical Record Generated
↓
SHA-256 Hash



Stored blockchain metadata includes:

- Record ID
- Hash
- Transaction hash
- Block number
- Contract address
- Timestamp
- Network/status

46. Hardhat

Hardhat is used as the blockchain development/testing environment.

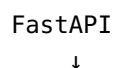
It is responsible for:

- Compiling Solidity
- Running the local blockchain network
- Deploying the smart contract
- Testing transactions
- Managing development accounts
- Providing a controlled blockchain environment

Ganache is not part of the architecture.

47. Web3.py Integration

FastAPI communicates with the smart contract through Web3.py.



```
Web3.py
  ↓
Hardhat RPC
  ↓
Solidity Smart Contract
```

Private keys must remain server-side and must never be exposed to the frontend.

48. Security Architecture

Security must be applied at multiple levels.

Authentication

- JWT-based authentication
- Secure password hashing
- Role-based permissions

Authorization

Every protected API must validate:

- authenticated user
- role
- ownership
- organizational relationship
- access rights

Data Security

- Secrets stored in environment variables
- Database credentials not hard-coded
- Blockchain private keys not exposed to browser
- Medical records not stored on blockchain

Sharing Security

Every record share requires valid token authorization.

Auditability

Important actions are recorded in `audit_logs`.

49. Environment Configuration

Sensitive configuration should be supplied through environment variables.

Examples:

```
DATABASE_URL
JWT_SECRET
BLOCKCHAIN_RPC_URL
CONTRACT_ADDRESS
BLOCKCHAIN_PRIVATE_KEY
```

No sensitive credentials should be hard-coded in source code.

50. Backend Architecture

Recommended structure:

```
FastAPI
|
├─ Authentication
├─ User Management
├─ Patient Management
├─ Doctor Management
├─ Hospital Management
├─ Medical Reports
├─ Prediction Service
├─ SHAP Service
├─ Record Service
├─ Sharing Service
├─ Blockchain Service
├─ Verification Service
├─ Model Registry Service
└─ Analytics Service
```

The backend should keep AI, database, blockchain and business-logic responsibilities separated.

51. Suggested API Groups

Authentication

```
POST /auth/register
POST /auth/login
GET  /auth/me
```

Patients

```
GET    /patients
POST   /patients
GET    /patients/{id}
PUT    /patients/{id}
```

Health Information

```
GET    /patients/{id}/health
POST   /patients/{id}/health
PUT    /patients/{id}/health
```

Medical Reports

```
GET    /patients/{id}/reports
POST   /patients/{id}/reports
DELETE /reports/{id}
```

Predictions

```
POST /predict/{disease}
GET  /predictions
GET  /predictions/{id}
```

SHAP

```
GET /predictions/{id}/explanation
```

Records

```
POST /records
GET /records
GET /records/{id}
```

Sharing

```
POST /access-tokens
GET /access-tokens
POST /records/{id}/share
GET /shared-records
GET /received-records
```

Blockchain

```
POST /blockchain/register
GET /blockchain/records/{record_id}
```

Verification

```
POST /records/{id}/verify
GET /verification-history
```

Admin

```
GET /admin/users
GET /admin/hospitals
GET /admin/models
GET /admin/training
GET /admin/analysis
GET /admin/blockchain
GET /admin/analytics
```

Exact endpoint naming can be finalized during implementation, but the responsibilities should remain separated.

52. Frontend Architecture

Recommended:

```
React
|
├─ Authentication
├─ Role-based routing
├─ Patient dashboard
├─ Doctor dashboard
├─ Hospital dashboard
├─ Admin dashboard
├─ Shared components
├─ Tables
├─ Forms
├─ Charts
├─ Record views
├─ Token management
└─ API service layer
```

The UI should consume backend APIs instead of embedding database or model logic into React.

53. UI/UX Design Direction

The application should look like a modern healthcare SaaS platform.

Visual direction

- Clean light background
- Navy/dark navigation
- Blue/teal primary accents
- Rounded cards
- Subtle shadows
- Clean typography
- Accessible forms
- Clear status badges
- Clear hierarchy
- Responsive layouts

Important UX principle

Healthcare information should be understandable before it is technical.

For example:

Preferred

Diabetes — High Risk — 87.4% Confidence

instead of:

Model Output Class 1 — Probability 0.874

Technical details can be available beneath the human-readable presentation.

54. Figma / Lovable Phase

The first development stage should focus on **UI/UX and information architecture**, not backend functionality.

The design process should include:

Phase 1 — Design System

Define:

- colors
- typography
- spacing
- buttons
- cards
- badges
- forms
- tables
- modals
- navigation
- chart style
- states

Phase 2 — Role-Based Screens

Create complete screen families for:

- Patient
- Doctor
- Hospital
- Admin

Phase 3 — User Flows

Design the complete flows before coding.

Examples:

```
Doctor Login
  ↓
New Patient
  ↓
New Analysis
  ↓
Prediction
  ↓
Record
```

and:

```
Patient
  ↓
Access & Sharing
  ↓
Generate Token
```

and:

```
Doctor A
  ↓
Record Sharing
  ↓
Token Validation
  ↓
Hospital B / Doctor B
```

and:

```
Hospital B
  ↓
Received Record
  ↓
Verify Integrity
```

↓
Verified / Modification Detected

55. Antigravity Implementation Phase

Once the UI is finalized, the complete working system should be implemented in Antigravity.

The implementation should not simply reproduce screenshots.

It should connect:

```
UI
↓
React
↓
API
↓
FastAPI
↓
PostgreSQL
↓
ML
↓
SHAP
↓
Blockchain
```

All buttons, forms, filters, searches, uploads, dashboards and workflows must perform real operations.

56. Recommended AI-Assisted Development Workflow

The project should be built in controlled phases.

Phase A — UI Generation

Use Figma/Lovable to establish:

- screen layouts

- navigation
- responsive structure
- visual hierarchy
- reusable components
- role-based pages

Phase B — UI Review

Before functionality is added, verify:

- no unnecessary patient tabs
- no model tabs for doctor
- disease names clearly visible
- record structure understandable
- token flow understandable
- received record source visible
- profile icons correctly positioned

Phase C — Antigravity Foundation

Implement:

- repository structure
- React frontend
- FastAPI backend
- PostgreSQL
- authentication
- RBAC
- environment configuration

Phase D — Healthcare Data Layer

Implement:

- patients
- doctors
- hospitals
- health profiles
- medical reports
- relationships

Phase E — AI/ML Integration

Integrate:

- existing trained model files

- disease-specific preprocessing
- model registry
- prediction APIs
- confidence/risk
- prediction storage

Phase F — Explainable AI

Implement:

- SHAP generation
- contribution data
- charts
- feature tables
- human-readable analysis

Phase G — Medical Records

Implement:

- record generation
- canonical representation
- record history
- report-to-prediction relationships

Phase H — Blockchain

Implement:

- Solidity contract
- Hardhat network
- Web3.py
- SHA-256 hashing
- blockchain registration
- transaction storage

Phase I — Consent and Sharing

Implement:

- access token creation
- expiration
- token validation
- record selection
- sharing
- receiving

Phase J — Verification

Implement:

- SHA-256 recalculation
- blockchain hash retrieval
- comparison
- verification result
- verification logs

Phase K — Admin Analytics

Implement dynamic:

- model metrics
- prediction statistics
- hospital statistics
- system activity
- blockchain metrics

Phase L — Testing

Perform full end-to-end testing.

57. Testing Strategy

Testing should cover four major layers.

Unit Testing

Test:

- preprocessing
- validation
- prediction functions
- SHAP calculations
- hash generation
- token validation

API Testing

Test:

- authentication

- patient creation
- prediction
- reports
- records
- sharing
- verification
- admin functions

Integration Testing

Test:

```
FastAPI
+
PostgreSQL
+
ML
+
SHAP
+
Blockchain
```

End-to-End Testing

Example:

```
Doctor creates patient
↓
Uploads report
↓
Runs Diabetes prediction
↓
SHAP generated
↓
Record created
↓
Hash registered
↓
Patient views record
↓
Patient generates token
↓
Doctor shares record
↓
Hospital receives
```

↓
Hospital verifies
↓
Integrity result displayed

58. Important Negative Test Cases

The project should explicitly test failure cases.

Sharing

- Invalid token
- Expired token
- Wrong patient token
- Wrong doctor
- Wrong hospital
- Record does not belong to patient

Authentication

- Invalid credentials
- Unauthorized role
- Disabled user

Blockchain

- Registration failure
- Network unavailable
- Contract unavailable

Verification

- Hash matches
- Hash mismatch
- Record missing
- Blockchain record missing

AI

- Missing required feature
- Invalid value
- Incorrect category
- Unsupported disease
- Model unavailable

59. Non-Functional Requirements

The system should aim to be:

Usable

Doctors and patients should understand the workflow without technical knowledge.

Secure

Unauthorized users must not gain access to protected healthcare data.

Reliable

Failures in AI or blockchain operations should be handled gracefully.

Maintainable

AI, blockchain, database and UI components should remain modular.

Scalable

The architecture should allow more diseases, models and hospitals to be added later.

Auditable

Important healthcare and sharing operations should leave an audit trail.

60. Error and State Handling

Every major screen should support:

Loading State

Example:

Loading patient records...

Empty State

Example:

No medical records found.

Validation State

Example:

HbA1c is required for this model.

Error State

Example:

Prediction service is temporarily unavailable.

Success State

Example:

Medical record created and registered successfully.

Verification State

Green:

Record Integrity Verified

Red:

Possible Modification Detected

61. Project Data Flow

The complete system data flow is:

```
Patient
↓
Doctor Creates Patient
↓
Clinical Data + Reports
↓
Disease Selected
↓
Disease-Specific ML Model
↓
Prediction
↓
```

```
Confidence / Risk
↓
SHAP
↓
AI Analysis
↓
Medical Record
↓
PostgreSQL
↓
Canonical Representation
↓
SHA-256
↓
Smart Contract
↓
Hardhat Blockchain
↓
Patient Views Record
↓
Patient Generates Consent Token
↓
Doctor Uses Token
↓
Record Sharing
↓
Receiving Hospital / Doctor
↓
Integrity Verification
↓
SHA-256 Comparison
↓
Verified / Possible Modification
```

62. What AI Does

AI/ML is responsible for:

- disease prediction
- probability/confidence where supported
- risk classification
- model evaluation
- SHAP explanation
- prediction analysis

AI does **not** control blockchain integrity.

63. What Blockchain Does

Blockchain is responsible for:

- storing the original record fingerprint
- maintaining an immutable registration
- supporting integrity verification

Blockchain does **not** store the complete patient medical record.

Blockchain also does not itself prevent an off-chain PostgreSQL record from being edited. Instead, the stored fingerprint provides an independent reference against which the current record can be verified.

64. What PostgreSQL Does

PostgreSQL stores the operational healthcare information:

- users
 - patients
 - doctors
 - hospitals
 - health profiles
 - reports
 - predictions
 - SHAP
 - medical records
 - tokens
 - sharing
 - verification history
 - model metadata
 - audit information
-

65. Core Innovation

The major innovation is not the individual technologies alone.

The contribution is their integration into a single workflow:

Machine Learning

Answers:

What disease risk does the model predict?

Explainable AI

Answers:

Why did the model make this prediction?

PostgreSQL

Answers:

Where is the complete medical record managed?

Patient Consent Token

Answers:

Who is allowed to receive/share the record and for how long?

SHA-256 + Blockchain

Answers:

Has the record changed from the version originally registered?

66. Final Role-Based Summary

Patient

Views and controls healthcare information.

Dashboard
My Health
My Records
Medical Reports
Access & Sharing
Profile

Doctor

Creates patients, performs analysis and manages records.

- Overview
- Patients
- New Patient
- New Analysis
- Record Sharing
- Records
- Profile

Hospital

Receives, manages and verifies authorized records.

- Dashboard
- Authorized Records
- Received Records
- Shared Records
- Verify Record
- Verification History
- Organization Profile

Admin

Manages the entire platform and AI/blockchain infrastructure.

- Dashboard
- Users
- Approvals
- Hospitals
- Patients
- Doctors
- AI Model Registry
- Model Training
- Model Analysis
- Medical Records
- Blockchain
- Verification Logs

67. Final End-to-End Use Case

Consider a patient named Riya.

Step 1 — Patient Registration

Doctor creates:

PAT-1025 — Riya Patel

Step 2 — Medical Information

Doctor enters the patient's clinical information and attaches relevant reports.

Step 3 — Disease Analysis

Doctor selects:

Diabetes

The system loads the actual Diabetes model configuration and required features.

Step 4 — AI Prediction

The trained model produces:

Diabetes — 87.4% Confidence — High Risk

Step 5 — SHAP

The system identifies the actual feature contributions.

Step 6 — Medical Record

The system generates REC-00125.

Step 7 — Database Storage

The complete record is stored in PostgreSQL.

Step 8 — Blockchain Registration

The record is canonicalized, hashed using SHA-256, and the hash is registered through the Solidity contract on Hardhat.

Step 9 — Patient Views Record

Riya opens My Records and sees:

- reports she provided
- data used
- disease prediction
- confidence
- risk
- analysis
- SHAP
- doctor
- hospital
- blockchain registration

Step 10 — Patient Creates Token

Riya selects:

Hospital A

Dr. A

6 hours

The system generates an access token.

Step 11 — Doctor A

Doctor A sees the active token inside Riya's patient profile.

Step 12 — Doctor A Shares

Doctor A selects:

Hospital B

Dr. B

Riya's token

Riya's specific medical record

The backend validates the token.

Step 13 — Hospital B Receives

Hospital B sees:

Received from: Hospital A / Dr. A

Step 14 — Integrity Verification

Hospital B clicks:

Verify Integrity

The system compares the current record's SHA-256 with the blockchain-registered hash.

Step 15 — Result

If equal:

Record Integrity Verified

If different:

Possible Modification Detected

68. Project Deliverables

The final project should contain:

Design

- Figma/Lovable UI design
- Design system
- Role-based flows
- Responsive screens

Frontend

- React application
- Role-based routing
- Responsive dashboards
- Tables/forms/charts
- Record viewers
- Token workflows

Backend

- FastAPI
- Authentication
- RBAC
- APIs
- Database services
- Prediction services
- Sharing services
- Verification services

AI/ML

- Trained model integration
- Model registry
- Disease-specific preprocessing
- Prediction pipeline
- Confidence/risk
- SHAP explanations
- Training/evaluation metadata

Blockchain

- Solidity smart contract
- Hardhat network
- Web3.py integration
- SHA-256 registration
- Verification

Database

- PostgreSQL schema
- Relationships
- Migrations
- Seed/sample data where appropriate

Testing

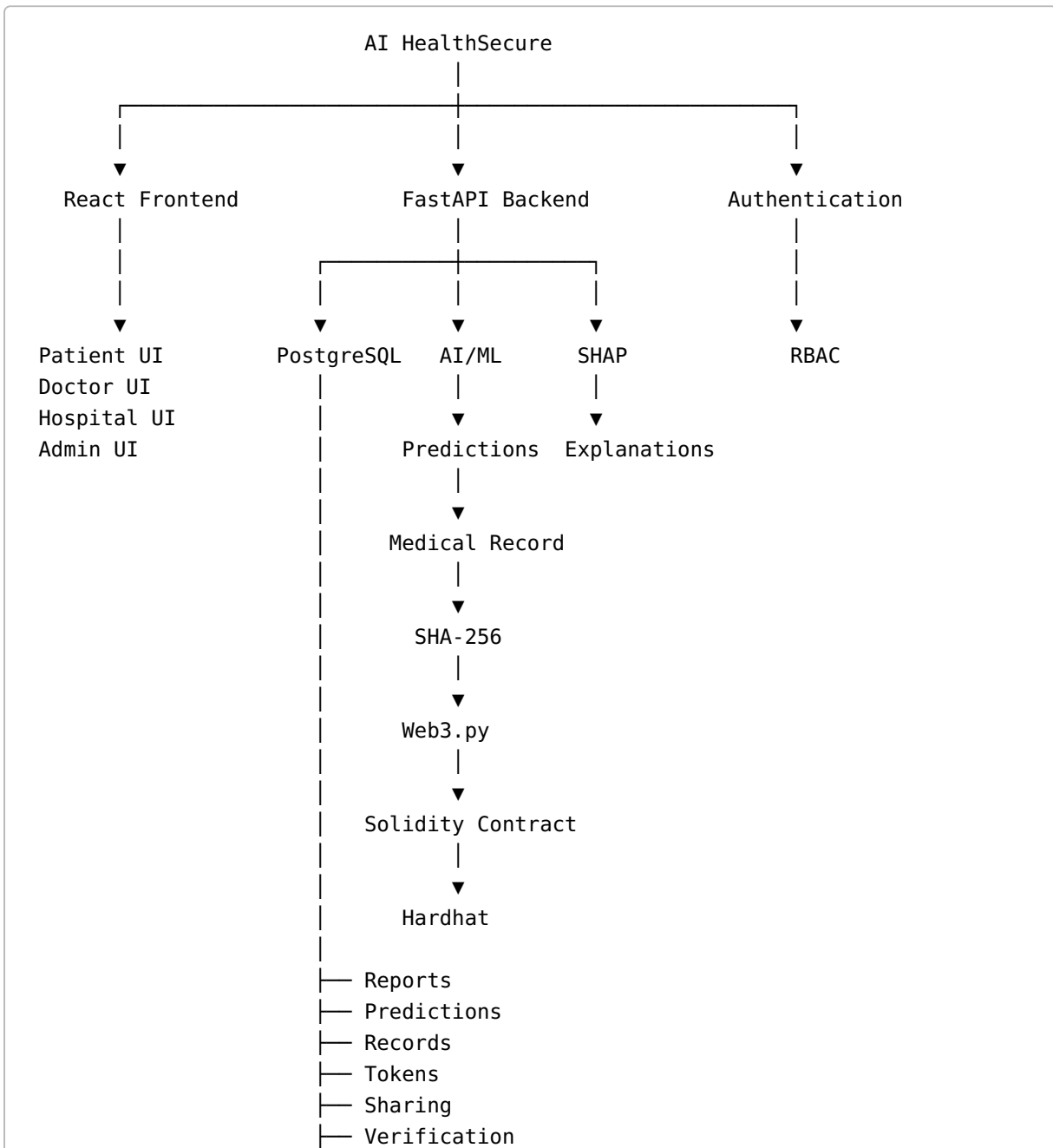
- Unit tests
- API tests
- Integration tests
- End-to-end tests

Documentation

- Architecture
- ER diagram
- API documentation
- AI/ML pipeline

- SHAP methodology
- Blockchain architecture
- Security model
- User workflows
- Deployment guide
- Testing report

69. Final System Architecture



70. Final Project Statement

AI HealthSecure is an integrated healthcare platform that combines disease prediction using trained machine learning models, explainable AI using SHAP, structured medical record management using PostgreSQL, patient-controlled token-based consent for record sharing, and blockchain-based integrity verification using SHA-256 fingerprints stored through a Solidity smart contract on a Hardhat network.

The platform separates responsibilities clearly:

Patients control and view their healthcare information.

Doctors perform disease analysis and generate medical records.

Hospitals receive, manage and verify shared records.

Administrators manage users, AI models, system analytics and blockchain activity.

The complete development strategy is:

```
Figma / Lovable
↓
UI/UX Finalization
↓
Design System + Role Flows
↓
Antigravity
↓
React + FastAPI
↓
PostgreSQL
↓
AI/ML + SHAP
↓
Medical Records
↓
SHA-256
↓
Solidity + Hardhat + Web3.py
↓
```

Consent + Sharing



Integrity Verification



Testing



Final Deployment

The final system should prioritize **correctness, explainability, security, traceability and usability**, while ensuring that every prediction, chart, model metric, medical record, token and verification result is derived from actual application data rather than static placeholder content.